TASK19: Mini Git System

AIM: To design a simple Git-like web application that tracks file changes, stores versions, and displays commit history using HTML, CSS, JavaScript, and Node.js.

ALGORITHM:

1. Start the program
2. Create project files (HTML, CSS, JS, server.js, db.js)
3. Design UI for entering file content
4. Capture user input using JavaScript
5. Send data to server using fetch API
6. Store versions in database (array)
7. Assign version number to each commit
8. Save commit message and timestamp
9. Create API to fetch commit history
10. Display commit history in UI
11. Allow viewing previous versions
12. Stop the program

PROGRAM:

INDEX.HTML:

```
<!DOCTYPE html>
<html>
<head>
   <title>Mini Git System</title>
   <link rel="stylesheet" href="style.css">
</head>
<body>
<div class="container">
```

```
    <h2>Mini Git System</h2>
    <textarea id="content" placeholder="Write your code..."></textarea>
    <input id="message" placeholder="Commit message">
    <button onclick="commit()">Commit</button>
    <h3>Commit History</h3>
    <ul id="history"></ul>
</div>
<script src="script.js"></script>
</body>
</html>
```

STYLE.CSS:

```
body {
    font-family: Arial;
    background: #1e1e2f;
    color: white;
    display:flex;
    justify-content:center;
    align-items:center;
    height:100vh;
}
.container {
    width: 400px;
    text-align: center;
}
textarea {
    width: 100%;
```

```
height: 80px;
    margin: 5px;
}
input, button {
    margin: 5px;
    padding: 8px;
}
li {
    background: #333;
    margin: 5px;
    padding: 5px;
}
```

SCRIPT.JS:

```
function commit() {
    const content = document.getElementById("content").value;
    const message = document.getElementById("message").value;
    fetch("/commit", {
        method: "POST",
        headers: {"Content-Type":"application/json"},
        body: JSON.stringify({ content, message })
    }).then(loadHistory);
}
function loadHistory() {
    fetch("/history")
    .then(res => res.json())
    .then(data => {
```

```
        let list = document.getElementById("history");
        list.innerHTML = "";
        data.forEach(c => {
            let li = document.createElement("li");
            li.innerText = `v${c.version}: ${c.message}`;
            list.appendChild(li);
        });
    });
}
loadHistory();
```

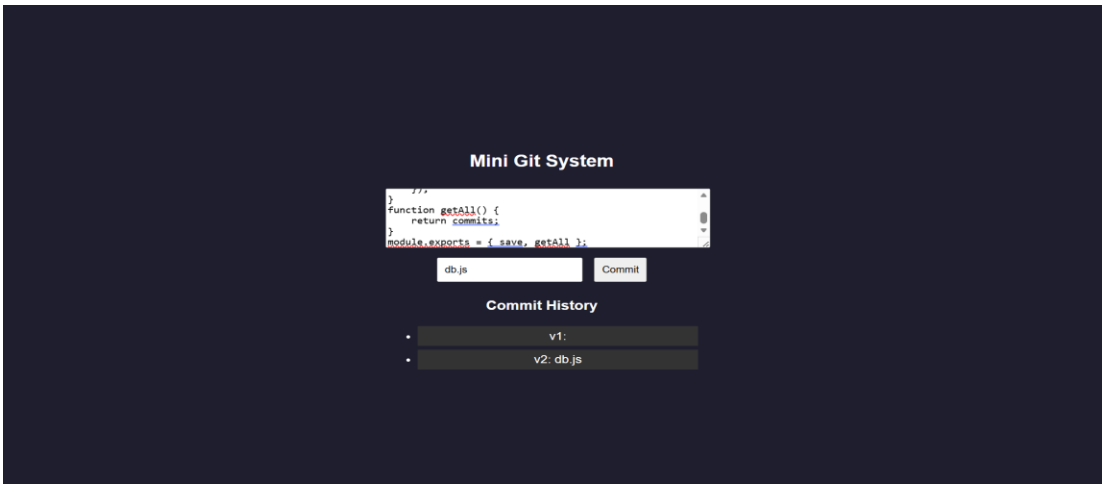
SERVER.JS:

```
const express = require("express");
const db = require("./db");
const app = express();
app.use(express.json());
app.use(express.static(__dirname));
app.post("/commit", (req, res) => {
    db.save(req.body);
    res.json({ msg: "Committed" });
});
app.get("/history", (req, res) => {
    res.json(db.getAll());
});
app.listen(3000, () => {
    console.log("http://localhost:3000");
});
```

DB.JS:

```
let commits = [];

let version = 1;

function save(data) {

   commits.push({

      version: version++,

      message: data.message,

      content: data.content,

      time: new Date()

   });

}

function getAll() {

   return commits;

}

module.exports = { save, getAll };
```

OUTPUT:



RESULT: A mini Git-like system was successfully developed to track versions, store commits, and display history through a web interface.